CA1 - Algorithms & Constructs: Fast Food Storage Java Application

Students: Caua Novais da Silva - 2023164
Tilek Ulan Uulu - 2023193
Yasmin Luize Nogueira Moura - 2023266

Useful links:

GitHub repository: https://github.com/CauaNovais/algorithms-constructs-ca1.git

Video link: https://www.youtube.com/watch?v=9zCswQbkKTE

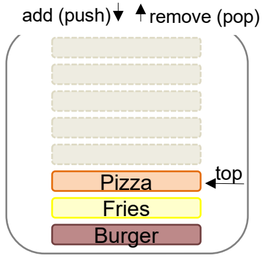
Lecturer: Taufique Ahmed

PROJECT OVERVIEW

The Food Storage Management System is a Java console application that manages up to eight food trays. Each item records its name, weight, best-before date (maximum 14 days) and storage time. Users can add, remove, view the top item, display all items and search by name.

WHY A STACK (LIFO)

According to Amaylia et al. (2025), in a stack the last item added is the first item removed (LIFO), using the push and pop operations, implemented as add and remove in our code. In this scenario, the chef can use the front side both to add and to remove trays, so a single opening naturally produces LIFO behaviour. A fixed-size array matches the eight positions, and push, pop and peek run in O(1) time. A queue would require access from both sides.



Array-based stack with eight positions.
Trays are added and removed only at the top.

ADVANTAGES OF USING LIFO

Simple management: Items are easy to add and remove.
Fast operations: Adding and removing items take O(1) time.
Controlled storage: The fixed-size array prevents the system from exceeding its capacity of eight items.

JAVA APPLICATION

The main class displays a menu with options from 0 to 5: add, remove, view top, display all, search and exit.

```
Food Storage: 0/8

1. Add food
2. Remove top food
3. View top food
4. Display all food
5. Search by name
0. Exit

Choose:
```

The storage operations are defined by the FoodStorage interface and implemented in the FoodStack class. The add method is shown below:

```
// Push operation: adds a tray to the top in O(1) time.
// Returns false if all 8 storage positions are occupied.
@Override
public boolean add(FoodItem item) {
    if (item == null) {
        throw new IllegalArgumentException("Food item is required.");
    }

    if (isFull()) {
        return false;
    }

    items[++top] = item;
    return true;
}
```

TIME COMPLEXITY

n is the number of trays currently stored.

Operation	Time	Reason
add (push)	O(1)	Places the tray at the next top index
remove (pop)	O(1)	Accesses only the top position
peek	O(1)	Returns the top without changes
display	O(n)	Visits every stored tray
search	O(1) best, O(n) worst	Found at the top, or at the bottom or absent

Space complexity: O(n), bounded by the fixed capacity of 8.

TESTING & ERROR HANDLING

Error handling was implemented for invalid inputs and empty storage. The examples below show how the application responds:

```
Food Storage: 0/8

1. Add food
2. Remove top food
3. View top food
4. Display all food
5. Search by name
0. Exit

Choose: 2
Storage is empty.
```

```
Choose: 1
Food name (Burger, Pizza, Fries, Sandwich or Hotdog): Pizza
Weight in grams: abc
Enter a valid number for weight.
```

```
Best-before date (YYYY-MM-DD): 2026-11-15
Best-before date must be from today to 14 days ahead.
```

When successfully added, the item will be displayed in a way the user will understand it was successfully added.

```
Choose: 1
Food name (Burger, Pizza, Fries, Sandwich or Hotdog): Burger
Weight in grams: 500
Best-before date (YYYY-MM-DD): 2026-11-05
Added: Burger | 500.0 g | Best before: 2026-11-05 | Stored at: 2026-11-01 01:46

Food Storage: 1/8
```

STORAGE CAPACITY

The storage holds a maximum of 8 items, controlled by a fixed-size array in the FoodStack class. When it is full, new items are rejected until one is removed.

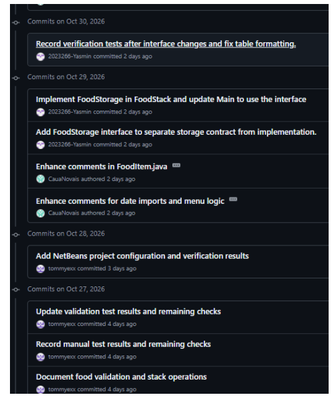
```
Choose: 1
Storage is full.

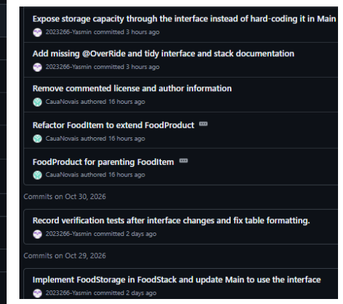
Food Storage: 8/8
```

LIMITATIONS

Older trays stay at the bottom and may expire; the 14-day limit reduces this risk.
Search may need to check every position, O(n).
Capacity and food names are fixed in the code.

COMMITS AND CONTRIBUTIONS:





Tilek (tommyexx): initial implementation, including the FoodItem model with validation, the array-based stack with LIFO operations, display and search, the console menu and the first manual tests.

Caua (CauaNovais): repository setup, FoodProduct parent class with FoodItem inheritance, code comments and the initial poster draft.

Yasmin (2023266-Yasmin): FoodStorage interface, capacity method, abstract FoodProduct, output formatting, final testing, README and the final poster.

REFERENCES

Amaylia, S., Setiabud, V.A., Renza Alvianino, Saputra, R.N., Kusuma Wardhani, H. and Azis Suroni (2025). Application of stack data structure in application development. Journal of Advanced Systems Intelligence and Cybersecurity, [online] 1(01). Available at: https://journal.unesa.ac.id/index.php/jasic/article/view/44459? (Accessed: 29 October 2026)